

Sr. No. _____

Date: _____

④ Backend Concepts → Node.js

→ Node.js runs JS code on one main thread.

→ one task at a time

→ no parallel execution.

→ fast, because it is nonblocking, starts an op, moves on, the comes back later

Heavy work goes to OS / thread pool → results come back via event loops and callback queue.

→ therefore Node.js appears concurrent without multiple threads

Limitation → CPU heavy tasks (ML, image processing)
Only one thread can get blocked

⑤ Event loop.

→ Node.js uses a event loop to handle async operations on a single thread.

Execution order → ① Call Stack (sync code)

② Microtasks

- process.nextTick (highest priority)
- promises (.then / await)

③ Macrotasks (event loop phases)

- timers (setTimeout, setInterval)
- I/O callbacks
- setImmediate
- close callbacks

process.nextTick > promises > setTimeout /
setImmediate

Evergreen

Sr. No. _____

Date: _____

① Event loop Phases

Phases order →

1. Timers
2. Pending callbacks
3. Idle/prepare
4. Poll (I/O happens here)
5. Check (setImmediate)
6. Close callbacks

② Inside I/O operations like

js. readfile (-file, {});
setTimeout
setImmediate};
setImmediate will run first, cuz I/O goes to poll, then check phase (setImmediate)

③ Microtask Queue

→ Runs immediately after current execution, before event loop continues

process.nextTick > Promise

* → nextTick dangerous → runs before everything, can starve the event loop.

Basically, nexttick maintains its own queue, so in a macrotask (event loop phase) a nexttick (microtask) has its own separate queue leading to infinite loops

fine loop {
 process.nextTick ~~setInterval~~ (loop);
 }
 loop();

infinite loop/
use setImmediate rather
setInterval.

Sr. No. _____

Date: _____

④ Standard streams (streams allow reading in chunks)

stdin	input	Duplex (read/write)
stdout	output (console.log())	Transform
stderr	error output (console.error())	Duplex that modifies

④ Module caching

require() loads a module only once and caches it.

Basically if a file counter.js exists, and we do,

```
const f1 = require("./counter");
const f2 = require("./counter");
```

module.export = {}
content++

```
console.log(f1()) // 1
console.log(f2()) // 2
```

} maintains memory.

module.exports is the object that's returned when you do a .require() a module

Force reload : delete ~~cache~~ require.cache [require.resolve("./utils")]

④ ASYNC Error handling (IMP)

→ try/catch does not work for async errors.

callback style:- // readFile is asynchronous, and this the format
fs.readFile("file.txt", (err, data) => {
 if (err) console.error(err);
});

for error detection in file reading

Sr. No. _____

Date: _____

Promise style:-

```
async function test() {  
  try {  
    await Promise.reject("error");  
  } catch(e) {  
    console.log("caught");  
  }  
}
```

Why try/catch fails?

→ because error happens later, outside call stack.

④ Sync → nextTick → Promise → timers → I/O → setImmediate

Node → single threaded

Async → event loop

modules cached

try/catch ≠ async safety

nextTick can block loop

⑤ Callback

→ Hell code → func call inside func call, hoops horrible syntax.

solution:-

⑥ Promises

- is an object that represents a future object that will exist later.

Sr. No. _____

Date: _____

A promise can be pending, fulfilled and rejected

Promises with then and catch:-

```
func fetchUser(id) {  
  return Promise { }
```

```
fetchUser(1)
```

```
  .then( user => {
```

```
    console.log(user);
```

```
    return user.name; // this gets passed to another .then
```

```
  })
```

```
  .then( name => {
```

```
    console.log(name);
```

```
  } )
```

```
  :
```

```
  .catch( error => {
```

```
    console.error( "Error"; error) // any rejection in chain
```

```
  .finally( () => {
```

```
    console.log( done) // runs always
```

A .then() returns another promise, allowing chaining

Promise.all([p1, p2, p3]) // all must complete

Promise.race([.....]) // first to complete

Promise.allSettled([]) // wait for all, regardless of success/fail.

Promise.any(?) → // any successful

Promise.resolve() → wrap value in promise

Promise.reject() → create immediate rejection

Sr. No. _____

Date: _____

④ Async/await → syntactic 'sugar' over promises.
→ simply changes syntax

Eg: fetchUser(1)

- then (user ⇒ fetchOrders (userid))
 - then (~~user.name~~ orders ⇒ ^{console.log} (orders))
 - catch ()
- } promises

async/await

```
async func getUserOrders() {  
  try {  
    const user = await fetchUser(1)  
    const orders = await fetchOrders (user.id)  
    console.log (orders)  
  }  
  catch {  
    console.error()  
  }  
}
```

async → wraps the function in a promise
allows you to use await inside

~~return type~~
console.log (asyncfunc()) ⇒ // Promise { "Fulfilled" : 'hello' }

await → pauses the async function
→ unwraps the resolves, gives you the resolved value. (imp)

to unwrap promises use .then() or await

write async/await inside try/catch

Sr. No. _____ Cases

Date: _____

1. Simple one off use promise with then
2. Multiple operations use async/await
3. multiple parallel operations → Promise.all
4. To handle success failure → use async await.

--dirname → ^{folder}~~file~~ path

--filename → file path

Http module:-

```
const http = require("http")
http.requirecreateServer((req, res) => {
  res.end("Hello");
}).listen(3000);
```

req → incoming request

res → outgoing response.

Node JS truly single threaded;

JS → single threaded

Background work → thread pool

④ Unhandled exception

→ when a exception is thrown, but there is not try/catch to handle that exception

Async (Promise) → error can be caught by .catch()

async (await) → can be caught with try/catch.